

Share Your Knowledge

If you are an expert in your field of electronics, why not share your knowledge with the world?



Electronics For You invites electronics professionals and industry experts to write articles on their area of expertise and interest from different perspectives

Some of the topics we want you to write about are:

- Photonics: Optics & Optoelectronics
- Automation
- Power Electronics
- Components
- Telecom

So if you have an interesting topic or original article to share, please let us know at efyeditorial@efy.in

We would love to hear your good ideas too!

electronics
FOR YOU

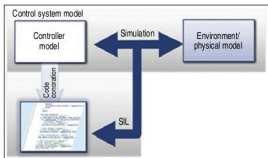


Fig. 2: Platform-independent testing in simulation

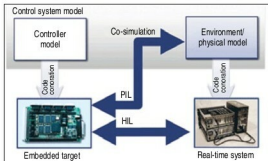


Fig. 3: Platform-dependent testing

lation. The behaviour of software functionality captured in simulation must meet the expectation specified in the requirements. Signal inputs and model outputs are represented as time-series data (sequences of data points in time). Typically, such test vectors can be derived from requirements, but test data can also come from measurements done on existing systems or from a model of the physical system that embedded software interacts with.

Testing methods where inputs of the software model are created with help from other models are sometimes referred to as model-in-the-loop testing. The term 'in loop' comes from control system applications with a feedback loop between an embedded controller model and the physical model being controlled.

An example would be a model of cruise-control software that controls a model of the vehicle engine. Such a system can also include a model of vehicle dynamics, test track and so on. An example of test data in this case can be a series of test vectors representing the driver's input in the cockpit of such a virtual vehicle.

Software-in-the-loop

Testing the software algorithm, where functions of generated or handwritten codes are evaluated in co-simulation on the host machine, is called software-in-the-loop (SIL) testing. As in model testing in simulation, input test vectors can come from requirements or other models in the executable specification. SIL tests typically reuse test data and model infrastructure used for model testing in simulation.

This type of verification is particularly useful when software components consist of a mix of generated codes (for example, updates to meet new requirements) and handwritten codes (for example, existing drivers and

data adapters) that may be necessary for execution on the target platform.

SIL testing is also used for verification of reimplementation of existing algorithms in graphical models. Some of the legacy codes, while correct, may be difficult and expensive to maintain, and it makes sense to reimplement and verify these in a graphical environment. In this case, models and simulations form the test framework for comparison of outputs of the new model implementation.

Processor-in-the-loop

A good starting point in verification of the compiled object codes on the target platform is verification that ensures functional equivalence of codes running on the target processor relative to the model behaviour captured in simulation.

Conceptually, processor-in-the-loop (PIL) testing is similar to SIL. The key difference is that during PIL, codes execute on the target processor or on an instruction set simulator. Data passed between the harness model and deployed object codes uses real input/output (I/O) via CAN, or serial devices.